# The hierarchy problem

A DRAM access can take many processor cycles. Sending every load and store directly to DRAM would leave a fast datapath waiting. A memory hierarchy combines levels of different capacity, cost, and latency to make most accesses appear fast while providing substantial capacity.

The benefit depends on locality. It is not a guarantee for arbitrary access patterns. Every model in this chapter identifies capacity, transfer unit, miss cost, and write policy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Address units and notation

A virtual address, VA, identifies a process location. Removing its page offset gives a virtual page number, VPN. Translation produces a physical address, PA. Removing the cache-block offset gives a physical block number.

Addresses count bytes unless stated otherwise. A word index must be scaled by four in this MIPS32 model. A block number is the byte address divided by block size. Let $B$ denote block bytes and $P$ denote page bytes when those symbols are used.

A correct-looking bit calculation with the wrong address unit still gives the wrong mapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.1 Locality and levels

Registers, L1, lower-level caches, DRAM, persistent storage, and remote storage trade increasing capacity for generally increasing latency and lower cost per byte.

Temporal locality means a recently used location is likely to be reused soon. Spatial locality means nearby locations are likely to be accessed. Consider consecutive 4-byte integers at byte addresses 0, 4, 8, 12, and 16. With an initially empty cache, 16-byte blocks, no prefetch, and no eviction, accesses are miss, hit, hit, hit, miss. The three hits exploit spatial locality. Reading 16 again while its block remains resident illustrates temporal locality.

Locality is a workload property exploited by a design, not a universal theorem about programs.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Average access time

Let $H$ be hit time, $m$ miss probability per access, and $P_m$ the additional miss penalty after the hit lookup.

$$AMAT = H + mP_m.$$

At $H = 1$, $m = 0.05$, and $P_m = 80$ cycles, AMAT is five cycles. Raising miss rate to 20% gives 17 cycles. Most references can be fast while the few misses dominate average time.

Distinguish misses per instruction from misses per data reference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The memory wall and design tradeoffs

Processor execution and DRAM latency do not improve at identical rates. Capacity, associativity, and block size can reduce selected misses while increasing lookup delay, fill traffic, area, or power.

L1 prioritizes short hit paths. Lower caches can afford larger capacity and associativity. DRAM supplies capacity and bandwidth. Persistent and remote levels accept much larger access costs.

A lower miss rate alone does not prove a faster hierarchy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.2 The basic cache model

A cache line contains data, a tag, and a valid bit. A write-back line also has a dirty bit. An access selects a set, compares valid tags, selects a matching way, and uses the offset to choose data.

On a miss, a controller obtains a block from the next level and chooses a location or victim. Valid prevents uninitialized tags from creating false hits. Dirty tracks a difference from the backing copy, not whether the tag matches.

Record data, tag, valid, and dirty changes separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Direct mapping

$$index = block_number \bmod N_{blocks}.$$

Each memory block has one possible cache position. For a power-of-two block count, the low block-number bits select it. Offset chooses within the block; tag supplies the remaining identity.

Frequently reused blocks mapping to the same position evict each other even if other positions are unused. Addresses separated by the cache's data capacity provide a useful conflict example.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A concrete access trace

A four-line direct-mapped cache, initially invalid, receives block numbers `0,1,2,3,0,4,0`.

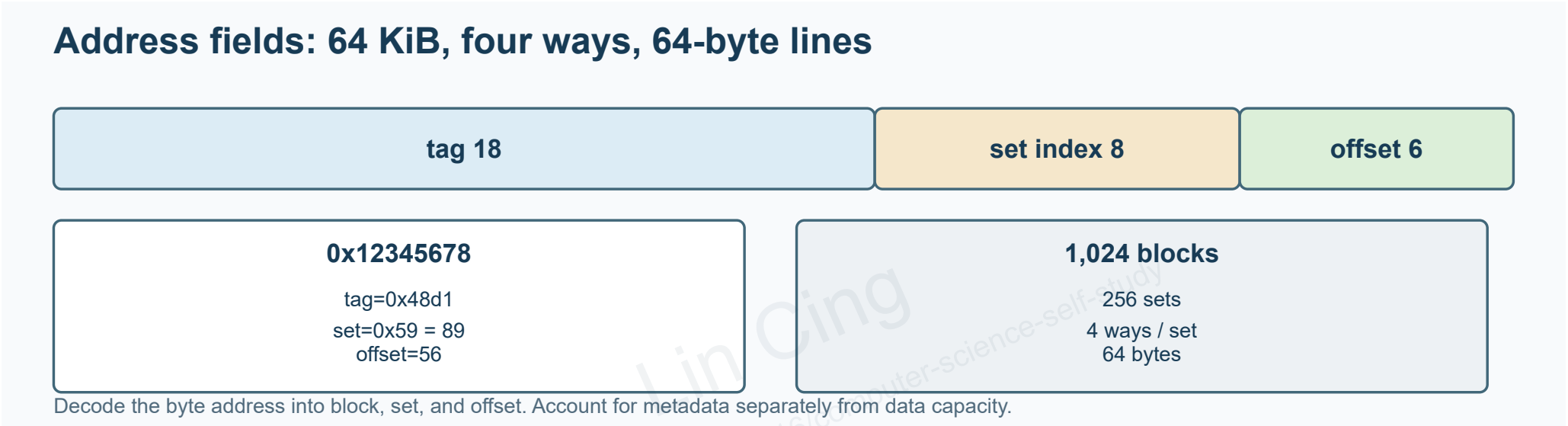
| Access | Block | Index | Result |
|---|---|---|---|
| 1 | 0 | 0 | Miss, initially invalid |
| 2 | 1 | 1 | Miss, initially invalid |
| 3 | 2 | 2 | Miss, initially invalid |
| 4 | 3 | 3 | Miss, initially invalid |
| 5 | 0 | 0 | Hit |
| 6 | 4 | 0 | Miss, evicts 0 |
| 7 | 0 | 0 | Miss, evicts 4 |

# A concrete access trace (continued)

There are six misses in seven accesses. First touches of 0–4 are compulsory. The final access is a conflict miss under the same-capacity fully associative LRU comparison, which would retain recently used block 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Address decoding for a 64-KiB four-way cache




With 64-byte lines and 32-bit addresses,

$$N_{blocks} = 2^{16}/2^{6} = 1024, \quad N_{sets} = 1024/4 = 256.$$

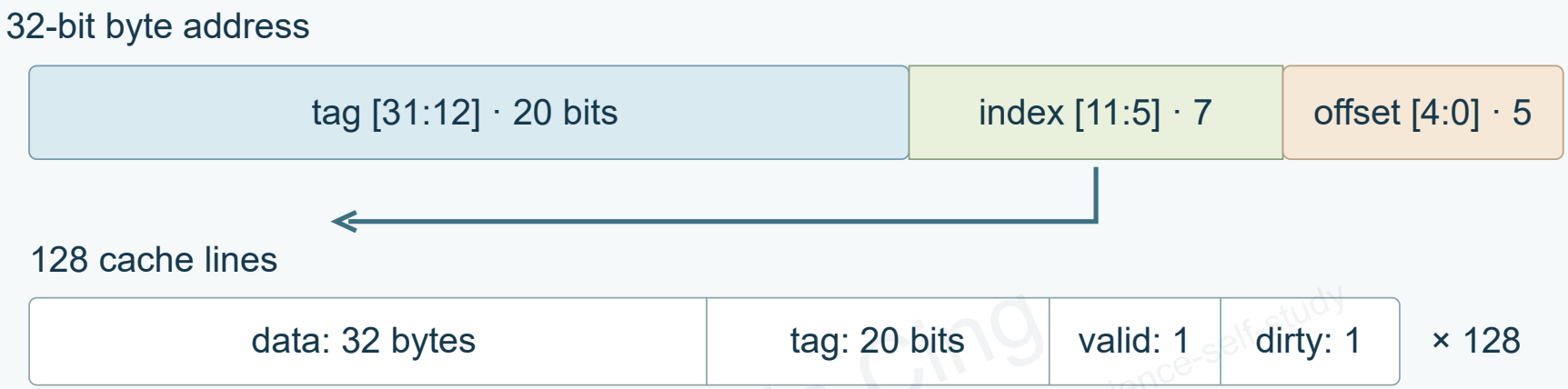
Offset has six bits, index eight, and tag $32 - 6 - 8 = 18$. For `0x12345678`, offset is `0x38` or 56, index is `0x59` or 89, and tag is `0x48d1`. Index bits are `01011001`.

# Address decoding for a 64-KiB four-way cache (continued)

`0x12345658` shares block base `0x12345640` and differs only in offset, so it hits if the block remains valid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Reconstruct capacity and metadata from address fields




For a direct-mapped write-back cache with 32-bit byte addresses, five offset bits give 32 bytes per line and seven index bits give 128 lines. Data capacity is therefore $128 \times 32 = 4096$ bytes.

Each line also holds a 20-bit tag, one valid bit and one dirty bit. Metadata occupies $128 \times 22 = 2816$ bits = 352 bytes. Counting only these fields, total storage is 4448 bytes, or 1.0859375 times the data capacity; ECC and other control circuitry are excluded.

# Handling misses

A blocking controller saves the request address, requests the lower-level data, handles a victim, installs data/tag/valid, and retries. An instruction miss needs the original request PC. A data miss needs the original address and any store data.

A dirty victim must be written back or safely buffered before its storage is overwritten. Critical-word-first can return the requested word before the entire fill completes.

A cache miss is ordinarily a hardware transaction and stall. A page fault invokes OS recovery and has different state requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Supporting a block fill

For four words, assume one address cycle, 15 cycles per memory activation, and one cycle per word transfer. A one-word-wide organization takes

$$1 + 4(15) + 4 = 65 \text{ bus cycles.}$$

A two-word-wide memory and bus can take $1 + 2(15) + 2 = 33$ cycles in this model. Widening only memory gives $1 + 2(15) + 4 = 35$. These are explicit transaction models, not universal DRAM timings.

Larger blocks exploit spatial locality but can increase fill cost and pollution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Interleaved memory: overlapping independent banks

Place four consecutive words in four independent banks. Assume one shared address operation starts all four in cycle 1, each bank takes 15 cycles, and the bus transfers one word per cycle. Once data is ready, four serial transfers complete the fill in 1 + 15 + 4 = 20 bus cycles.

Compare 65 cycles for the single-bank model: the gain comes from overlapping bank latency, not transferring four words at once on a narrow bus. The result changes if addresses cannot be issued together, accesses collide in a bank, or responses require additional spacing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Handling writes

Write-through propagates each write to the next level, usually through a buffer. Write-back changes the cached copy and marks it dirty, propagating later on eviction or protocol demand.

Write allocation installs a missed block before updating it. No-write-allocation forwards the store without installing the line. Allocation and propagation are independent decisions.

A partial-word or partial-line update needs byte enables or an equivalent cache-array update mechanism. It does not inherently require a main-memory read-modify-write transaction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Write-policy boundary cases

| Policy | Write-miss action | Main consequence |
|---|---|---|
| Through, no allocation | Forward store | Low allocation cost, high write traffic |
| Through, allocation | Fetch, update both levels | Possible later hits, additional traffic |
| Back, allocation | Fetch, update and dirty | Coalesces repeated writes, dirty victims |
| Back, no allocation | Forward missed store | Avoids pollution, misses later reuse |

A store buffer must also supply or order a following read of an address whose write is pending. Returning an older cache value would violate the required memory semantics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

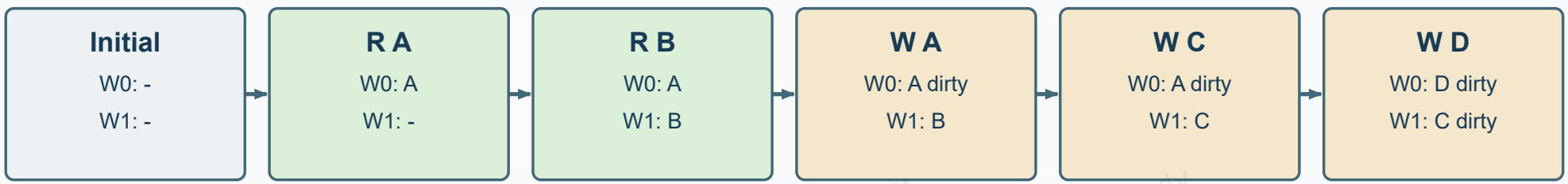
# Write-back, allocation, and dirty victims

Use a two-way set, 64-byte blocks, four-byte stores, LRU, write-back, and write-allocate. A/B/C/D map to this set. Both ways start invalid; choose the lower-numbered empty way. Ways are fixed physical slots; LRU updates do not move lines.

| Step | Access | Way 0 | Way 1 | Victim and lower-level action |
|---|---|---|---|---|
| 0 | initial | invalid | invalid | none |
| 1 | read A | A clean | invalid | read A block |
| 2 | read B | A clean | B clean | read B block |
| 3 | write A | A dirty, MRU | B clean | hit, no lower write |
| 4 | write C | A dirty | C dirty, MRU | discard clean B, read C, then modify C |
| 5 | write D | D dirty, MRU | C dirty | write back dirty LRU A, then read D |

# Write-back, allocation, and dirty victims (continued)

## Two-way write-back cache: LRU and dirty victims




**W C evicts clean B. W D evicts dirty A; write A back before installing D.**

**Lower traffic: read A/B/C/D = 4 blocks; write-back A = 1 block. The A write hit stays in cache.**

Four blocks are read and A alone is written back. Overwriting A before preserving its newest value loses committed data. Dirty does not affect hit detection, but determines whether eviction must preserve a newer value. Correct write-back alone is not a persistence guarantee: cache and DRAM contents may both be lost on power failure.

# Write-back, allocation, and dirty victims (continued)

With write-through/no-write-allocate, write A immediately updates the lower level. Writes C and D do not allocate or evict B. The final set retains A/B, with two block reads and three word writes.

Counting payload only, no headers/acks or final flush, WB/WA moves $4(64) + 1(64) = 320$ bytes. WT/NA moves $2(64) + 3(4) = 140$ bytes. Both have five lower-level actions but different transfer sizes. Flushing final dirty C/D would add 128 bytes to WB; persistence still depends on platform flush/ordering guarantees.

A write hit requires exclusive ownership through the store. Write-back delays the lower update without delaying the cache's program-order value. Victim storage can be reused only after its data enters memory, a reliable buffer, or another owner with guaranteed completion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The FastMATH case

The historical Intrinsity FastMATH example separates instruction and data caches. Each 16-KiB cache contains 4096 words in 16-word blocks, giving 256 blocks.

Under direct mapping, index is eight bits and byte offset six. Capacity alone does not establish every timing or protocol detail. Split caches remove a shared IF/MEM port conflict but cannot pool unused instruction capacity for data.

Keep historical implementation numbers separate from current measurements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.3 Cache performance and CPI

$$T_{CPU} = (C_{execution} + C_{stall})T_{clock}.$$

For a blocking model with instruction miss rate $m_i$, data references per instruction $f_d$, data miss rate $m_d$, and common extra penalty $P_m$,

$$CPI = CPI_{base} + m_i P_m + f_d m_d P_m.$$

Out-of-order execution and memory-level parallelism can overlap costs. Do not apply full-stall addition without checking that assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# An AMAT and CPI example

One-cycle hits, 6% misses, and 80-cycle additional penalty give $AMAT = 1 + 0.06(80) = 5.8$ cycles.

Separately, with base CPI 1.4, instruction miss rate 2%, 0.25 data references/instruction, and data miss rate 8%, stalls add

$$0.02(80) + 0.25(0.08)(80) = 3.2.$$

Total CPI is 4.6. One billion instructions at 2 GHz take 2.3 s under this full-stall model.

Data miss rate applies to data references rather than every instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Flexible placement

For associativity $E$, $N_{sets} = N_{blocks}/E$. Direct mapping has one way. Full associativity has one set.

At constant data capacity, doubling associativity halves sets, removes an index bit, and adds a tag bit. Each selected set needs more parallel comparisons and selection. Reduced conflicts can come with higher hit time and energy.

Equal data capacity does not imply equal tag-array cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Finding a block and choosing a victim

Select a set, compare valid tags, and select the matching way. Multiple valid matches for the same block indicate an invariant failure in this model.

Random, LRU, and approximations choose victims. Two-way LRU needs one recency bit; exact LRU becomes more expensive at higher associativity. In a two-way set containing A and B, accessing B makes A the next LRU victim.

Replacement normally affects performance. Coherence and dirty-data handling impose additional correctness obligations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Multilevel AMAT denominators

Let $T_\ell$ be hit service time when level $\ell$ is reached, $M_\ell^{local}$ its miss fraction among requests sent there, and $P_{mem}$ the extra memory penalty after the last miss.

$$AMAT_1 = T_1 + M_1 T_2 + M_1 M_2^{local} P_{mem}.$$

For $T_1 = 1$, $M_1 = 0.04$, $T_2 = 8$, $M_2^{local} = 0.25$, and $P_{mem} = 100$,

$$AMAT = 1 + 0.04(8) + 0.04(0.25)(100) = 1 + 0.32 + 1 = 2.32.$$

| Quantity | Denominator | Value |
|---|---|---|
| M1 | all L1 references | 0.04 |
| L2 local miss | L1 misses only | 0.25 |
| L2 global miss | all L1 references | 0.01 |
| L2 hit fraction | L1 misses only | 0.75 |

Seventy-five percent of L1 misses pay only eight L2 cycles; one percent of all references additionally pay memory. Treating 25% as global gives the wrong $1 + 0.04(8) + 0.25(100) = 26.32$.

# Multilevel AMAT denominators (continued)

If $P_{2,miss,total} = T_2 + P_{mem} = 108$, this is conditional on also missing L2. L2 hits still cost eight. Conditional averaging is necessary:

$$AMAT_1 = T_1 + M_1[(1 - M_2^{local})T_2 + M_2^{local}P_{2,miss,total}] = 1 + 0.04[0.75(8) + 0.25(108)] = 2.32.$$

Only the expected cost per L1 miss,

$$\overline{P}_{1,miss} = T_2 + M_2^{local}P_{mem} = 33,$$

permits $AMAT_1 = 1 + 0.04(33)$. Multiplying conditional 108 directly by the L1 miss rate wrongly gives 5.32. Including L2 time in a penalty does not remove the need to distinguish its hit and miss cases.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.4 Why virtual memory exists

Virtual memory maps a process's addresses to physical frames, supports protection and controlled sharing, and simplifies allocation and relocation. It can also keep only active pages resident while backing other content with storage.

Address-space size is not physical residency. Historical overlays made software arrange code movement explicitly; paging transfers much of this work to the OS and hardware.

Virtual memory provides an address and protection contract, not unlimited fast storage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Page tables and translation

With 4-KiB pages, a 32-bit VA has a 20-bit VPN and 12-bit offset. A PTE can hold a physical page number, present/valid state, permissions, dirty/reference information, and caching attributes. Exact fields are platform-specific.

$$PA = PTE.PPN \parallel VA.offset.$$

The page offset is copied unchanged because a virtual page and physical frame have the same internal size and layout.

A missing page's backing location is typically interpreted through OS-managed metadata rather than assumed identical across PTE formats.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Page faults are more than cache misses

A TLB miss can occur while the page is resident. A fault can indicate absent content or denied access. For a recoverable not-present fault, save the address and context, obtain a frame, preserve a dirty victim, load or initialize content, update mappings, invalidate stale translations, and retry.

A major fault involving storage is expensive. Minor faults such as demand-zero allocation or copy-on-write need not access disk. Permission faults do not automatically authorize loading and retrying.

A faulting store must not leave an unauthorized partial architectural update.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Virtual-memory writes

Paging ordinarily uses write-back because storage cannot absorb a transaction for every processor store. A dirty page differs from its backing copy; a clean victim can often be discarded without writing.

An invalid PTE may mean unallocated, swapped, or specially managed content. The OS combines fault address, access type, and mapping metadata to decide whether recovery is legal.

A dirty bit describes synchronization with backing state. It is not a timestamp for the last write.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fast translation with a TLB

A TLB caches translations and permission information. Its tag can combine VPN and address-space identifier, ASID. A hit provides the PPN without walking the full page table.

A miss triggers hardware or software refill, not necessarily a page fault. TLBs are small and often highly associative. Permissions and dirty/reference management must remain consistent with PTE changes.

A context switch can flush translations or retain them under ASID tags.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

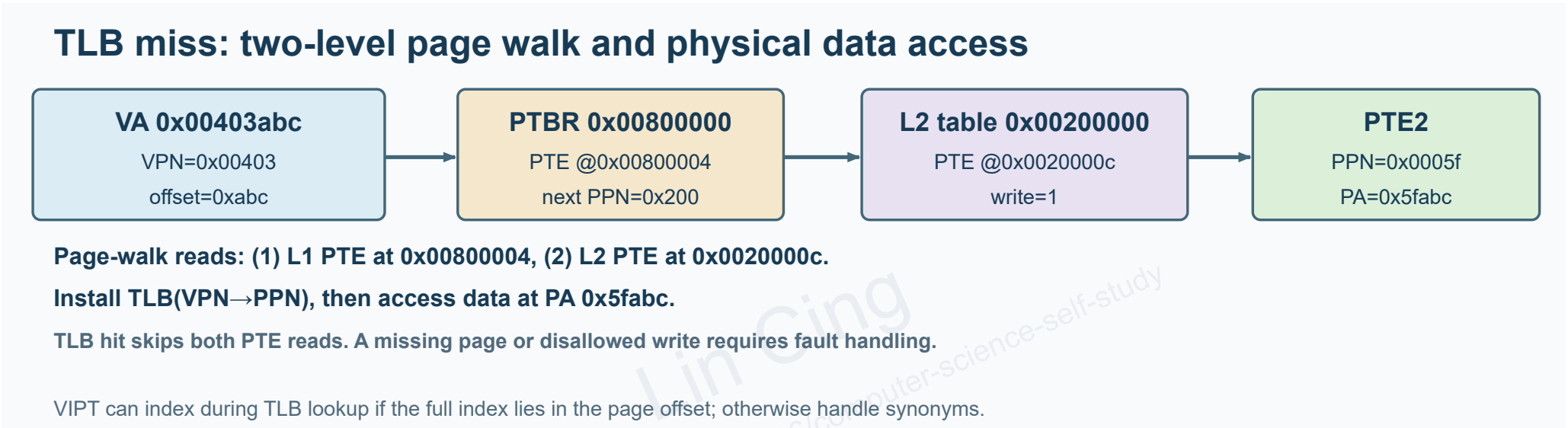
# A TLB miss, two-level walk, and cache lookup

Use 32-bit VA, 4 KiB pages, four-byte PTEs, and 1024 entries per level. `VA=0x00403abc` gives `VPN=0x00403` and offset `0xabc`. The ten-bit indices are L1=1, L2=3. The following PTE contents are explicit model inputs.

| Step | Address calculation | Read or decision |
|---|---|---|
| 1 | `PTBR=0x00800000`, add `0x001*4` | read L1 PTE at `0x00800004` |
| 2 | L1 PPN `0x00200` gives table base `0x00200000` | read L2 PTE at `0x00200000+0x003*4=0x0020000c` |
| 3 | L2 `present=1`, `write=1`, `PPN=0x0005f` | install `VPN 0x00403→PPN 0x0005f` |
| 4 | `PA=(0x0005f<<12)\|0xabc` | access data at `0x5fabc` |

https://github.com/xinhi6716/computer-science-self-study

# A TLB miss, two-level walk, and cache lookup (continued)




In this example, a miss without a walk-cache hit requires two PTE accesses plus data-cache lookup/fill. Page-table accesses themselves can hit/miss caches; dedicated walk caches change the count. A TLB hit skips both PTE reads. Valid resident PTEs mean a translation miss. A missing present bit causes a not-present page fault. Present 1/write 0 on a store means protection fault.

# A TLB miss, two-level walk, and cache lookup (continued)

A VIPT cache can index from the untranslated page offset while the TLB finds the physical tag. A simple safe configuration requires

$$\text{block-offset bits} + \text{index bits} \leq \text{page-offset bits}, \qquad BS \leq P,$$

or data capacity/ways≤page size. Comparing index width alone omits block offset.

For 64 KiB, four ways, 64-byte blocks, there are 256 sets. Offset bits 5:0 and index 13:6 total 14>12. Aliases VA `0` and `0x1000` for the same physical page/offset select sets 0 and 64. Sixteen ways gives 64 sets and 6+6=12, so these aliases select the same set.

A larger virtual index requires coloring, synonym maintenance, or PIPT. Comparing only PA[31:14] would omit physical bits 13:12 and allow a false hit. Comparing the full 20-bit PPN prevents this false identification, but cannot by itself eliminate duplicate copies in different sets. The 18-bit PIPT tag is not a sufficient physical identifier for this VIPT configuration.

# Protection and context changes

Read, write, execute, and user/supervisor permissions must be enforced on a TLB hit as well as during a page-table walk.

Flushing on a context switch prevents a new process from using another process's same-VPN translation. ASIDs allow coexistence, but the OS must still handle identifier reuse, mapping changes, and invalidation.

Identity tags never replace access-permission checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Miss and fault recovery paths

| Event | Evidence | Response |
| --- | --- | --- |
| Cache miss | No valid matching tag | Fill lower-level data |
| Resident TLB miss | No TLB match, valid permitted PTE | Refill translation |
| Not-present page | PTE indicates absent content | OS recovery if mapping is valid |
| Protection fault | Access permission denied | OS policy, signal, termination, or legitimate repair |
| Undefined VA | No valid mapping | OS fault handling |

Retries require precise or otherwise recoverable state. Sharing a fault-address register does not make these events equivalent.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 5.5 Four common questions

Where may a block reside? How is it found? Which victim is selected? How do writes propagate?

Caches, TLBs, and paging answer these with different placement, lookup, replacement, and write policies. Page-table lookup costs additional accesses but supports flexible physical-frame placement. Full associative placement is not free parallel associative lookup hardware for the entire page table.

Separate the freedom to place an object from the mechanism used to locate it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Three miss types require an explicit reference

Use a fully associative LRU shadow with the same block size and data capacity, updated on every reference. First establish that the actual cache misses, then classify:

| Type | Condition for this miss | Common remedy | Possible cost |
|---|---|---|---|
| Compulsory | First-ever access to this block | Larger blocks, prefetch | Traffic, pollution |
| Capacity | Previously accessed; shadow misses | More capacity, better locality | Hit time, cost |
| Conflict | Previously accessed; shadow hits | More associativity | Comparison and selection cost |

A first touch takes precedence as compulsory. Classification is per access, for a specified trace and reference policy. A working-set slogan or a replacement in the same line cannot classify every later miss.

# Classify three miss types using one shared trace

Use 32-bit byte addresses, 64 B of data capacity, 16 B blocks, an initially empty cache, and reads only: no prefetching or coherence invalidations. The actual cache is four-line direct-mapped (DM). Its reference is an equally sized, four-block fully associative **LRU shadow cache**.

```
block sequence: 0, 4, 0, 1, 2, 3, 4, 0, 1
byte addresses: 0x0000, 0x0040, 0x0000, 0x0010, 0x0020,
                0x0030, 0x0040, 0x0000, 0x0010
```

Every access uses offset zero. DM has four offset bits, two index bits, and 26 tag bits: `line=block mod 4`, `tag=floor(block/4)`. Blocks 0 and 4 have different tags but compete for line 0. The shadow instead permits any block in any of its four positions.

Check the actual cache first. A hit is not a 3C miss. On a miss, classify a first touch as compulsory; for later touches, use a shadow hit/miss to classify conflict/capacity. **Both caches process every access, including actual hits.** An LRU hit also updates recency.

# The third access conflicts despite unused capacity

**Same capacity: 4 blocks × 16 B = 64 B; initially empty**

### #1 block 0 (0x0000)

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | — | — | — |

**DM miss / FA miss**

First touch of this block

### #2 block 4 (0x0040)

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 4 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | 4 | — | — |

**DM miss / FA miss**

First touch of this block

### #3 block 0 (0x0000)

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 4 | 0 | — | — |

**DM miss / FA hit**

Conflict: 4 had displaced 0

DM rows have fixed line numbers. The FA boxes show LRU-to-MRU order, not fixed way numbers. Access 2 forces block 4 to displace block 0 in DM line 0. FA has spare space and retains both. Access 3 revisits block 0: DM misses while the shadow hits, making this a conflict miss.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Accesses 1–5: compulsory and conflict misses

States are **after** each access. A dash means invalid. H/M is the current hit/miss; FA entries run from LRU to MRU.

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM class |
|---|---|---|---|---|---|---|
| 1 | 0 | `[0,–,–,–]` | M | `[0]` | M | compulsory |
| 2 | 4 | `[4,–,–,–]` | M | `[0,4]` | M | compulsory |
| 3 | 0 | `[0,–,–,–]` | M | `[4,0]` | H | conflict |
| 4 | 1 | `[0,1,–,–]` | M | `[4,0,1]` | M | compulsory |
| 5 | 2 | `[0,1,2,–]` | M | `[4,0,1,2]` | M | compulsory |

# Accesses 6–9: distinguish capacity misses from hits

Access 6 leaves four blocks in the shadow. Continue the same state, without restarting from empty.

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM class |
|---|---|---|---|---|---|---|
| 6 | 3 | `[0,1,2,3]` | M | `[0,1,2,3]` | M | compulsory |
| 7 | 4 | `[4,1,2,3]` | M | `[1,2,3,4]` | M | capacity |
| 8 | 0 | `[0,1,2,3]` | M | `[2,3,4,0]` | M | capacity |
| 9 | 1 | `[0,1,2,3]` | H | `[3,4,0,1]` | M | hit |

DM has eight misses: five compulsory, one conflict, and two capacity. Accesses 7 and 8 revisit blocks and miss in the shadow too. Replacement in DM line 0 alone does not make them conflict misses.

# A shadow miss need not be an actual miss

### #9 Read block 1 (byte address 0x0010)

**Direct-mapped**

Fixed lines 0 / 1 / 2 / 3

| block 0 | block 1 | block 2 | block 3 |
|---|---|---|---|

**HIT: block 1 remained in line 1**

State unchanged; this is not a miss.

Accesses #7 and #8 replace only line 0.

**Fully associative LRU shadow**

Before access: LRU → MRU

| block 2 | block 3 | block 4 | block 0 |
|---|---|---|---|

**MISS: access #8 evicted block 1**

Evict 2 → new order [3, 4, 0, 1]

Update the shadow, but count no actual miss.

Access 9 is an **actual hit and shadow miss**, contributing no actual miss. Both caches total eight misses: DM miss/FA hit at access 3 is canceled by DM hit/FA miss at access 9. A zero difference in totals does not erase the conflict. Classification must follow individual accesses.

# Two ways change replacement, even at the same capacity

With 64 B and 16 B blocks, a two-way cache has two sets, `set=block mod 2`, each with its own LRU order. Its pattern is `M M H M M M M M H`: seven misses. Before access 9, set 0 is `[4,0]` and set 1 is `[1,3]`, both LRU-to-MRU, so block 1 still hits.

| Design | Hits | Misses | Hit time H | Total time (additional miss penalty = 20 cycles) |
|---|---|---|---|---|
| DM | 1 | 8 | 1 cycle | `9×1+8×20=169 cycles` |
| Two-way LRU | 2 | 7 | 1.25 cycles | `9×1.25+7×20=151.25 cycles` |

Every reference pays H; a miss pays the additional penalty. All references are serialized, without overlap. For this trace, two-way is faster exactly when `9×1.25+7P < 9×1+8P`, or `P>2.25 cycles`.

FA offers more placement freedom, but **LRU is not an optimal minimum-miss replacement algorithm**. Two-way has one fewer miss on this trace. More legal placements do not make every replacement policy better on every trace.

# 5.6 System virtual machines

A system VM presents an environment in which a guest OS can execute. The VMM manages privileged state, translation, interrupts, and devices while isolating guests.

Sensitive operations must be safely trapped, translated, or directly managed by virtualization support. Guest virtual, guest physical, and host physical addresses are different levels.

A process VM such as a JVM does not present the same complete system-ISA/device contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# VM translation and I/O

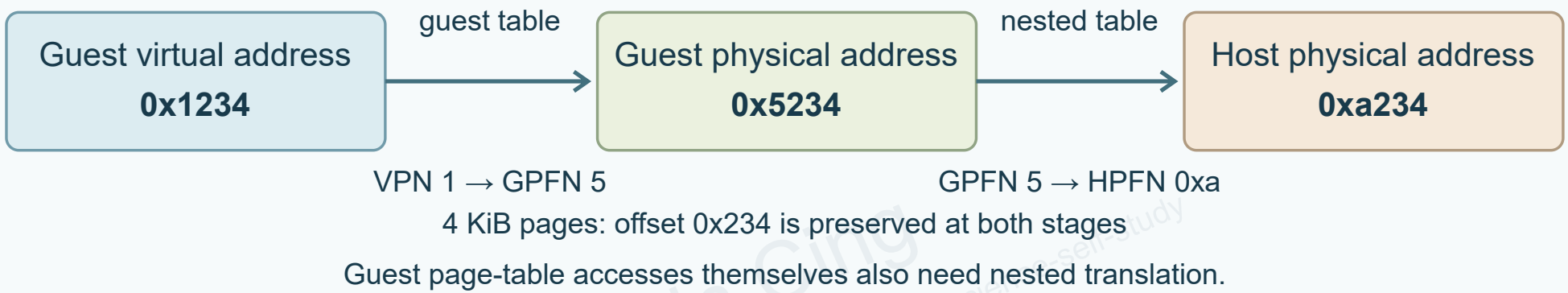
Shadow page tables can map guest VA directly to host PA while the VMM tracks guest table changes. Hardware nested paging instead performs composed walks, reducing some traps while increasing work on a translation miss.

Device emulation, passthrough, and IOMMUs make different isolation and performance tradeoffs. DMA, interrupts, queues, and ownership all need virtualization rules.

Evaluate which state is trapped or isolated before estimating the overhead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Nested paging: translating one address




Assume 4 KiB pages at both stages, valid mappings, and permitted access. A guest instruction emits GVA `0x1234`: guest VPN `1` and offset `0x234`. The guest page table maps VPN `1` to guest physical frame `5`, producing GPA `0x5234`.

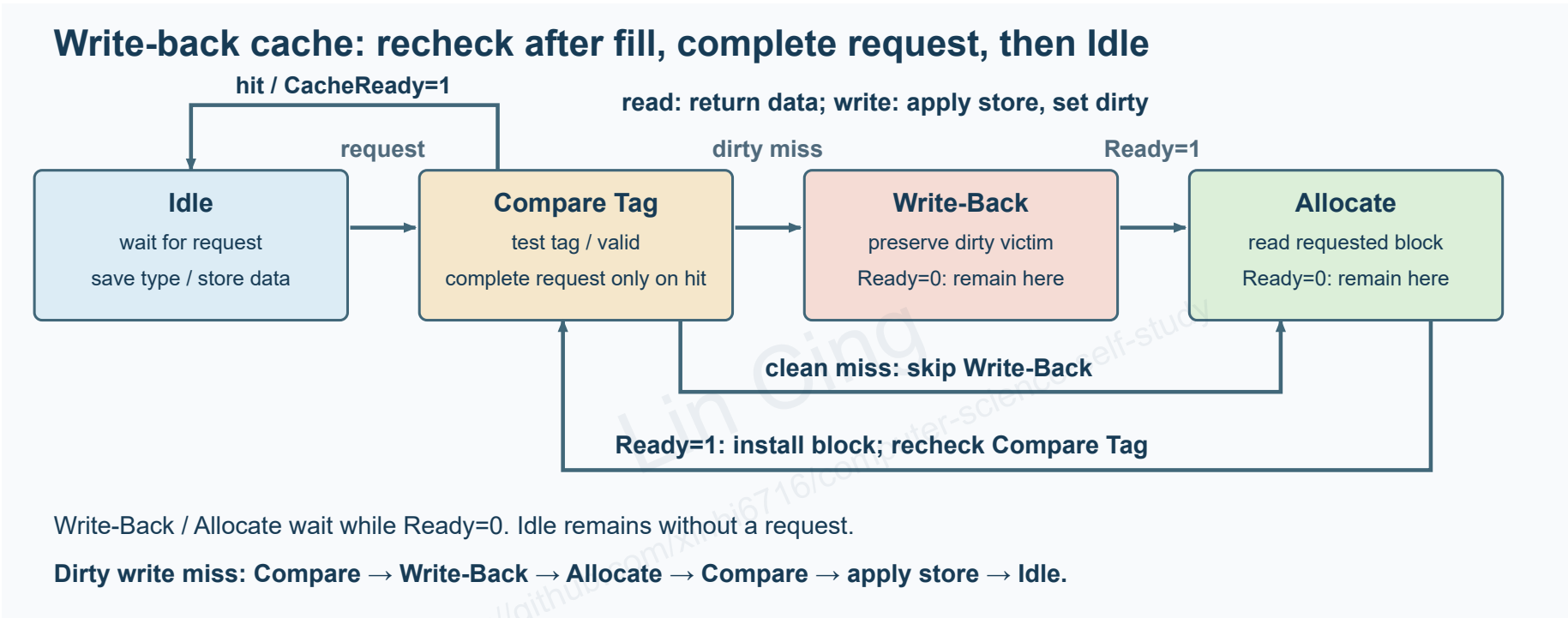
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The extra page-table translations

The nested table maps guest frame `5` to host frame `a`, producing host PA `0xa234`; both stages preserve the offset. The guest page table itself resides in guest physical memory, so reading a guest PTE also requires nested translation of that PTE’s GPA. TLB and walk-cache hits can reduce actual memory reads; no universal fixed cost follows from this example.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.7 Cache-controller state machine




Idle accepts a request. Compare Tag decides hit/miss and updates a write hit. Write-Back preserves a dirty victim while waiting for memory `Ready`. Allocate fetches the new block and returns to comparison after installation.

# 5.7 Cache-controller state machine (continued)

Variable memory latency requires handshake-controlled waiting. Unasserted enables default inactive, but selectors and transaction data still need defined behavior.

Write-back must complete or become safely buffered before allocation overwrites the victim.

After Allocate receives Ready, install the block, clear its initial dirty state, and return to Compare Tag. The recheck completes the original saved read or write; only then assert CacheReady and return to Idle. Write-Back and Allocate hold their states while Ready=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# State-transition traces

```
Clean read miss:
Idle -> Compare Tag -> Allocate -> Compare Tag -> Idle
                         | !Ready: remain here

Dirty write miss:
Idle -> Compare Tag -> Write-Back -> Allocate -> Compare Tag -> Idle
                        !Ready       !Ready
```

Save the original read/write type, address, and store data while waiting. The final comparison must complete the original request rather than confuse a store with a load.

Early restart and buffering add concurrency and therefore add state invariants.

# 5.8 Cache coherence

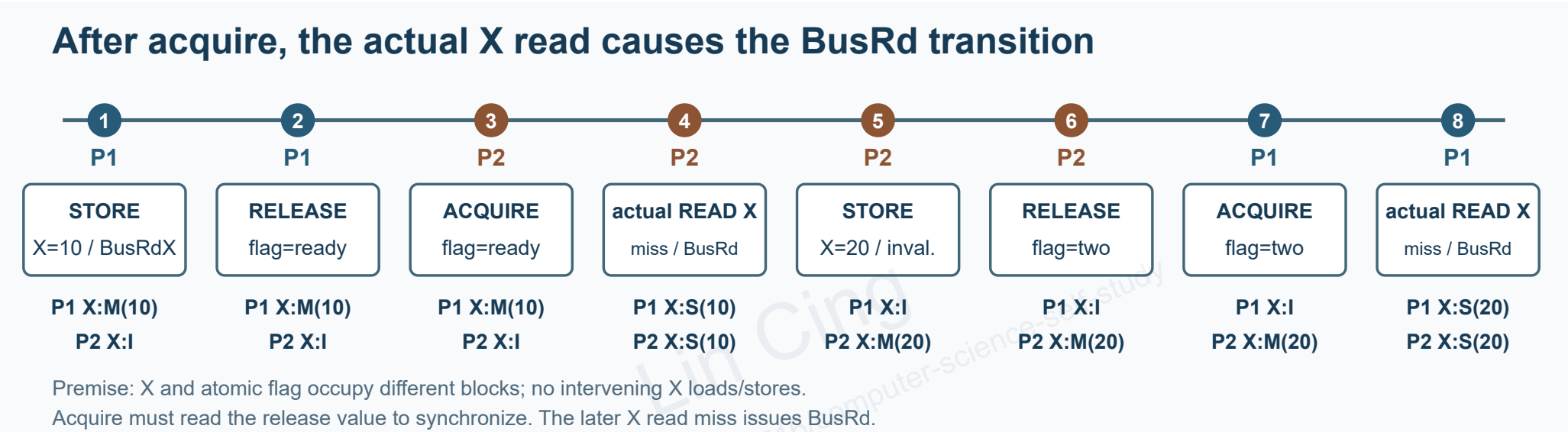
Private caches can hold multiple copies of one physical address. Coherence specifies read values for ordered accesses and a common serialization of writes to one location. A stale read during a data race does not by itself prove a protocol error; the consistency model determines which ordering exists.

The required properties include a processor reading its own earlier write when no intervening effective write to that address occurs, all processors observing writes to one address in the same order, and a synchronized read returning the preceding ordered write when no later same-address write intervenes.

Consistency defines cross-location and synchronization ordering; coherence regulates one location's copies and values. An atomic operation removes a particular race only if its specified synchronization covers those conflicting accesses. It does not repair unrelated unordered accesses.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

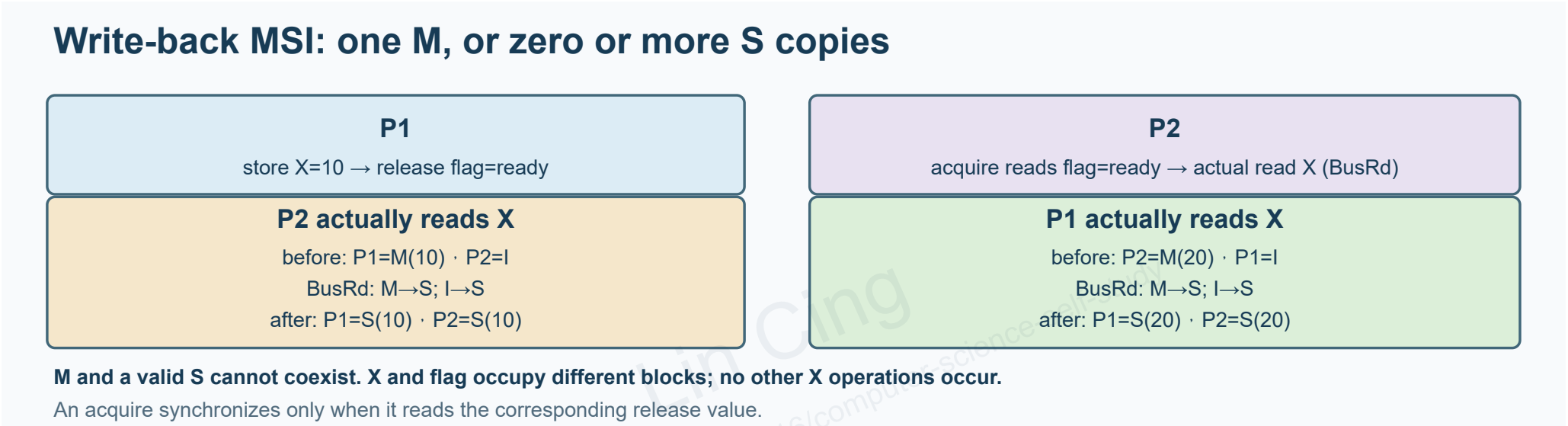
# Write-back MSI after intervening writes




P1 writes 10 and releases `ready`. P2 must actually read that released value with acquire before its later read of X; acquire itself does not read X. The X read miss issues BusRd and the M owner supplies 10.

P2 then writes 20 and releases `two`. After P1 acquires that value, its subsequent read obtains 20. The example places X and flag in separate blocks and assumes no other X accesses between these events.

# MSI invariants and acquire versus data reads




Assume X and atomic flag occupy different cache blocks, and no additional X operations occur between the listed events. A release/acquire synchronizes only when the acquire actually reads the released flag value.

# MSI invariants and acquire versus data reads (continued)

| Event | P1 X | P2 X | Ordering and memory consequence |
|---|---|---|---|
| initial | I | I | X not cached, no flag synchronization |
| P1 store X=10, BusRdX | M(10) | I | local cache updated, no cross-thread HB yet |
| P1 release flag=ready | M(10) | I | orders prior X store; dirty X need not flush |
| P2 acquire reads ready | M(10) | I | synchronizes with release; no X access/state change |
| P2 actually reads X, BusRd | S(10) | S(10) | M owner supplies 10, downgrades; memory becomes 10 |
| P2 store X=20, upgrade | I | M(20) | P2 obtains exclusivity and invalidates P1 |
| P2 release flag=two | I | M(20) | orders X=20 before flag write |
| P1 acquire reads two | I | M(20) | synchronizes; P1 has not read X yet |
| P1 actually reads X, BusRd | S(20) | S(20) | P2 supplies 20, downgrades; memory becomes 20 |

# MSI invariants and acquire versus data reads (continued)

At most one cache can hold M, and when M exists no other valid S exists. Each read miss obtains the newest value from the M owner and downgrades that owner to S. Acquire on flag is not a load of X. The actual X load causes BusRd. Sharing X's cache block with flag would add false-sharing traffic excluded by the premise here.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Snooping and write invalidation

Snooping caches observe broadcast coherence traffic. Scaling a broadcast domain costs bandwidth. A write-invalidate protocol obtains exclusive ownership before modifying a line.

| MSI state | Meaning |
|---|---|
| I | Invalid |
| S | Readable shared copy, possibly replicated |
| M | Sole valid cached copy; memory may be stale |

A write from S invalidates other copies. A remote read of M requires the owner to supply the newest data. Basic MSI returns both caches to S and makes memory current; a dirty shared owner needs an extended protocol such as MOESI.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.9 Designing a cache controller

A cache controller handles variable-latency memory with a state machine. Idle accepts a request; Compare Tag determines hit or miss; Write-Back preserves a dirty victim; Allocate waits for the lower-level block. While memory has not responded, `Ready=0`. On completion, install the data and tag, set valid, clear the new line's initial dirty bit, and return to Compare Tag. Only the rechecked hit that completes the original read or write asserts `CacheReady`. For a saved write, apply its store data and set dirty before returning to Idle.

Retain request type, store data, fault/retry context, and victim ownership across waits. Before overwriting the victim slot, Write-Back must complete or move its contents into a buffer with guaranteed completion. Allocation cannot bypass that ordering requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.9 Designing a cache controller (continued)

Useful invariants are that each valid line's tag identifies its data, dirty contents are preserved before their only storage is overwritten, responses occur only when the requested data is ready, and reset/error handling leaves no half-completed transaction. Assertions or model checking can test these stated invariants. The complete referenced HDL is unavailable in the supplied PDF; this is an explicitly specified original controller model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.10 Historical systems

The Barcelona Opteron X4 and Nehalem examples combine on-die caches, TLBs, and memory controllers. Private L1/L2 and shared lower-level storage reduce some off-chip communication while adding coherence, area, and power costs.

The examples illustrate sharing, inclusion, victim relationships, and intercore traffic. They are historical configurations, not current product rankings. An L2 miss can hit a noninclusive L3; noninclusion does not make that impossible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.11 Fallacies and pitfalls

For a 32-byte direct-mapped cache with four-byte blocks, byte address 36 denotes block 9 and index $9 \bmod 8 = 1$. Capacity is not the block count.

Do not mix local and global miss denominators, instructions and data references, or time and cycles. A larger L1 can improve hit rate yet lose because its hit path is slower.

Write units and denominators before inserting numerical rates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 5.12 The common hierarchy model

Locality, block transfer, placement, search, replacement, write propagation, and miss cost connect caches with paging. Their parameter scales differ greatly.

Multiple levels distribute competing objectives. Software blocking, loop ordering, and prefetching can improve locality but require workload measurements. Increasing L3 alone does not resolve bandwidth, coherence, or scheduling costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.13 Historical perspective

Limited memory, relocation, and protection motivated virtual memory and caching. Paging, TLBs, set associativity, and coherence became recurring solutions.

Complex implementations still preserve architectural contracts for process addresses and memory effects. Locality, protection, and movement costs remain relevant when technology ratios change.

Historical device values are evidence about a particular design period, not immutable ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# End-of-chapter check

Decode byte addresses, trace hits and dirty victims, derive AMAT and stall CPI, distinguish translation and data faults, and trace an invalidation protocol. State 3 C and consistency-model boundaries.

Use exercises independently, then inspect solutions. Every result needs the placement, timing, and write assumptions that make it valid.

## References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 5. CSO course slides, memory hierarchy.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study